第五章 动态规划法

Chapter 5 Dynamic Programming

计算机学院 高翔
gaoxg@nwpu.edu.cn

5.1动态规划概述

5.2 求解三角形最小路径问题

5.3求解最长公共子序列问题

5.4求解最长递增子序列问题

5.5求解0/1背包问题

5.6完全加括号的矩阵连乘积

5.7 动态规划算法的基本要素

5.8 求解资源分配问题

5.1 动态规划概述

5.1.1 从求解斐波那契数列看动态规划法

求解斐波那契数列的递归算法

```
int count=1;                    //累计调用的步骤
int Fib(int n)                  //算法
{  printf("(%d)求解Fib(%d)\n", count++, n);
   if (n==1 || n==2)
   {  printf("   计算出Fib(%d)=%d\n", n, 1);
      return 1;
   }
   else
   {  int x=Fib(n-1);
      int y=Fib(n-2);
      printf("   计算出Fib(%d)=Fib(%d)+Fib(%d)=%d\n",
                                        n, n-1, n-2, x+y);
      return x+y;
   }
}
```

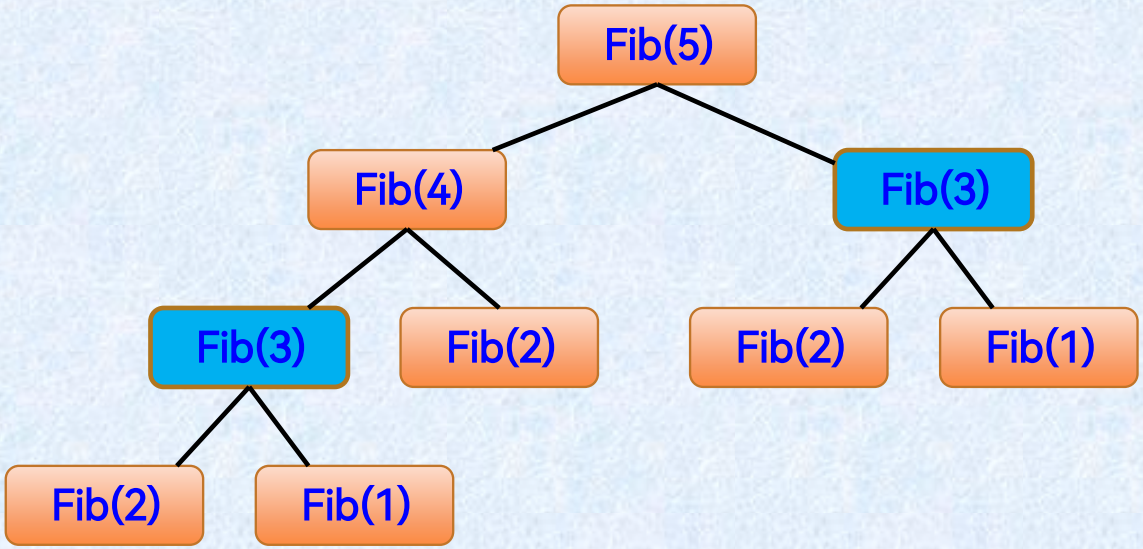
Fib(5)时的输出结果：

(1) 求解Fib(5)
(2) 求解Fib(4)
(3) 求解Fib(3)
(4) 求解Fib(2)
计算出Fib(2)=1
(5) 求解Fib(1)
计算出Fib(1)=1
计算出Fib(3)=Fib(2)+Fib(1)=2
(6) 求解Fib(2)
计算出Fib(2)=1
计算出Fib(4)=Fib(3)+Fib(2)=3
(7) 求解Fib(3)
(8) 求解Fib(2)
计算出Fib(2)=1
(9) 求解Fib(1)
计算出Fib(1)=1
计算出Fib(3)=Fib(2)+Fib(1)=2
计算出Fib(5)=Fib(4)+Fib(3)=5

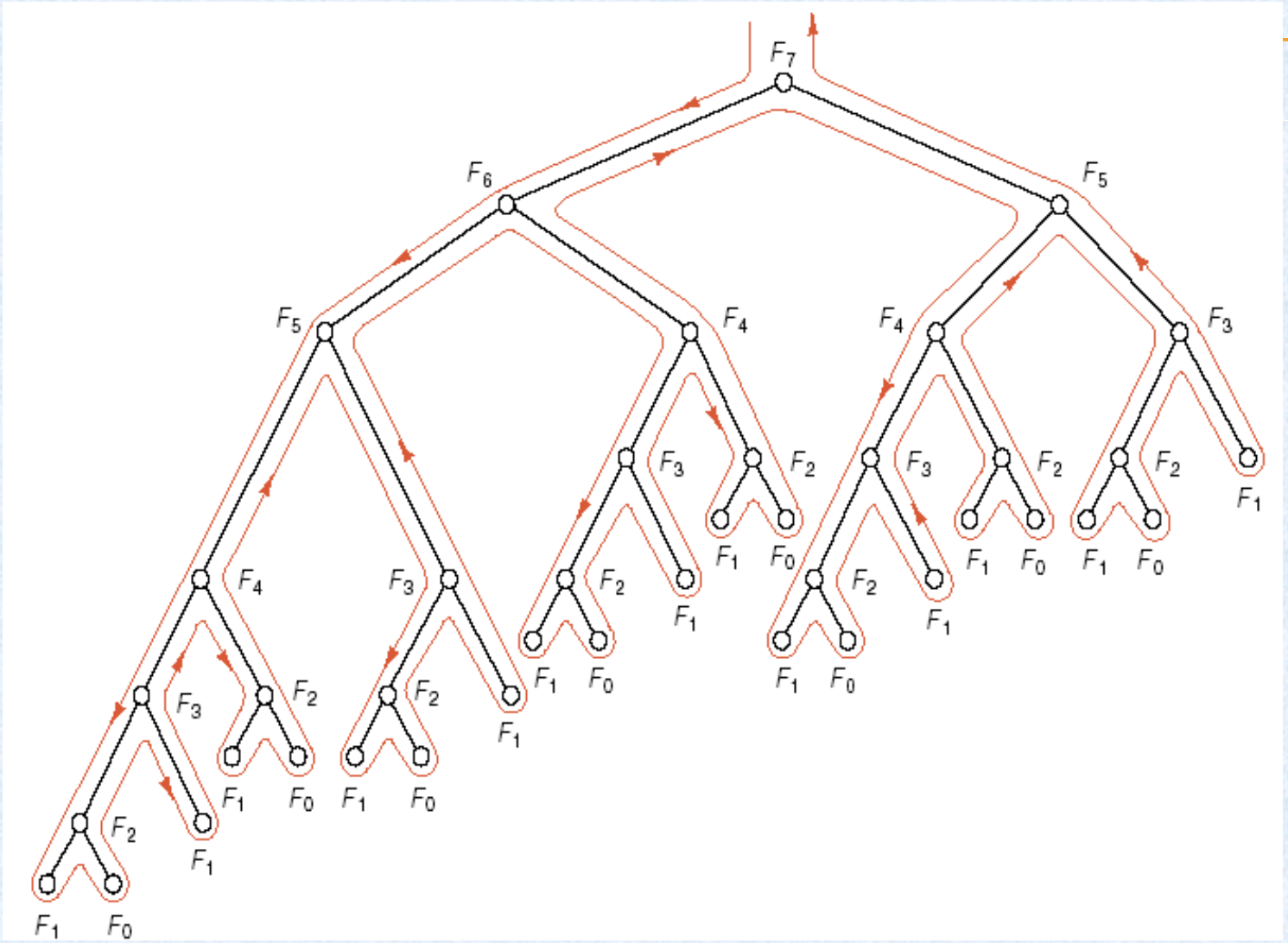
从中看出如下几点：

（1）递归调用Fib(5)采用自顶向下的执行过程，从调用Fib(5)开始到计算出Fib(5)结束。

（2）计算过程中存在大量的重复计算，例如求Fib(5)的过程如图8.1所示，存在两次重复计算Fib(3)值的情况。



5.2 斐波那契额数列


F_7
F_6
F_5
F_5
F_4
F_4
F_3
F_4
F_3
F_3
F_2
F_3
F_2
F_2
F_2
F_1
F_1
F_0
F_1
F_0
F_1
F_0
F_1
F_0
F_2
F_2
F_2
F_2
F_1
F_1
F_1
F_1
F_0
F_1
F_0
F_1
F_0
F_3
F_2
F_2
F_1
F_1
F_0
F_1
F_0
F_2
F_1
F_1
F_0

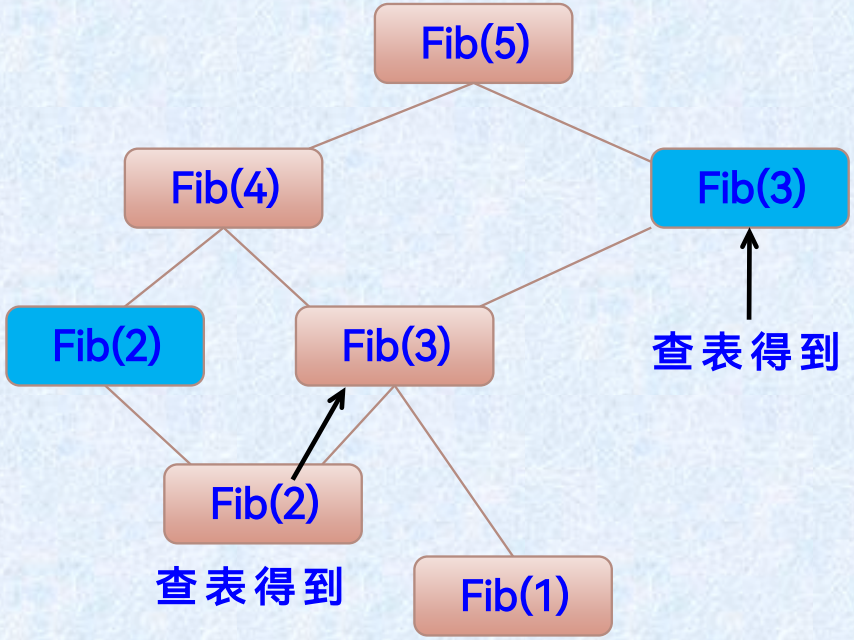
为此避免重复设计，设计一个dp数组，dp[*i*]存放Fib(*i*)的值，首先设置dp[1]和dp[2]均为1，再让*i*从3到*n*循环以计算dp[3]到dp[*n*]的值，最后返回dp[*n*]即Fib1(*n*)。对应的算法1如下：

```
int dp[MAX];                                    //所有元素初始化为0
int count=1;                                    //累计调用的步骤
int Fib1(int n)                          //算法1
{  dp[1]=dp[2]=1;
   printf("(%d)计算出Fib(1)=1\n"，count++);
   printf("(%d)计算出Fib(2)=1\n"，count++);
   for (int i=3;i<=n;i++)
   {  dp[i]=dp[i-1]+dp[i-2];
      printf("(%d)计算出Fib(%d)=%d\n"，count++，i，dp[i]);
   }
   return dp[n];
}
```

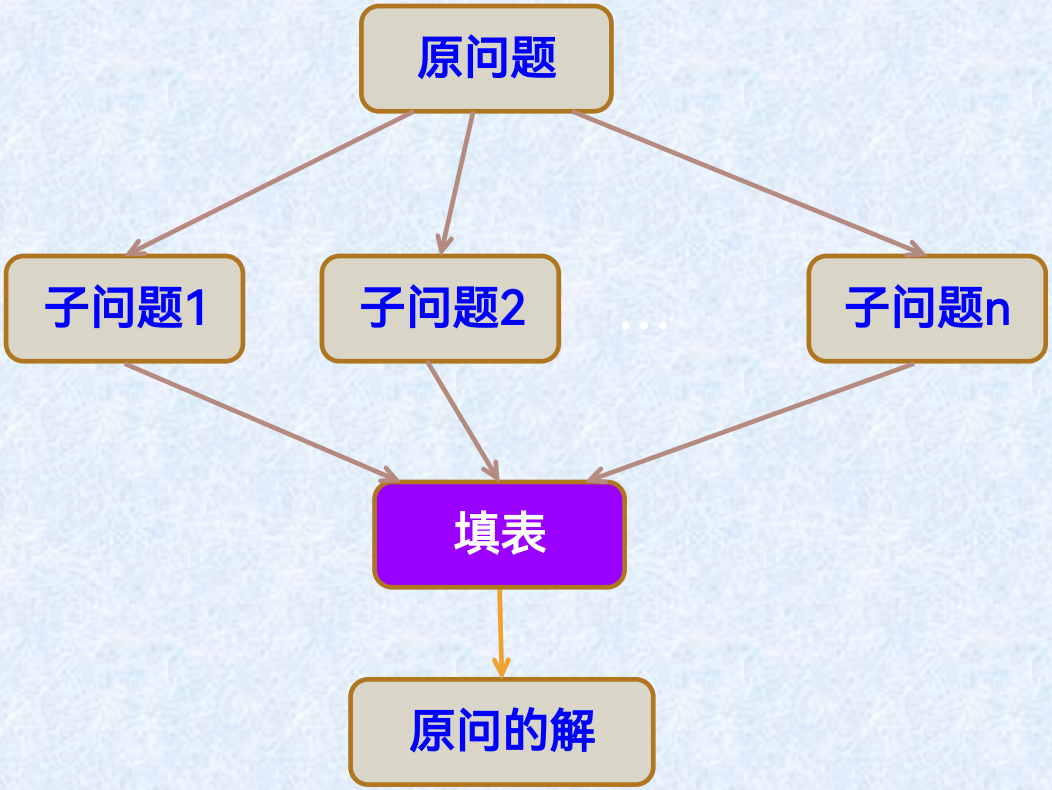
执行Fib(5)时的输出结果如下：

(1) 计算出Fib(1)=1
(2) 计算出Fib(2)=1
(3) 计算出Fib(3)=2
(4) 计算出Fib(4)=3
(5) 计算出Fib(5)=5

其执行过程改变为自底向上，即先求出子问题解，将计算结果存放在一张表中，而且相同的子问题只计算一次，在后面需要时只有简单查表，以避免大量的重复计算。



上述求斐波那契数列的算法1属于动态规划法，其中数组dp（表）称为动态规划数组。动态规划法也称为记录结果再利用的方法，其基本求解过程如下图所示。

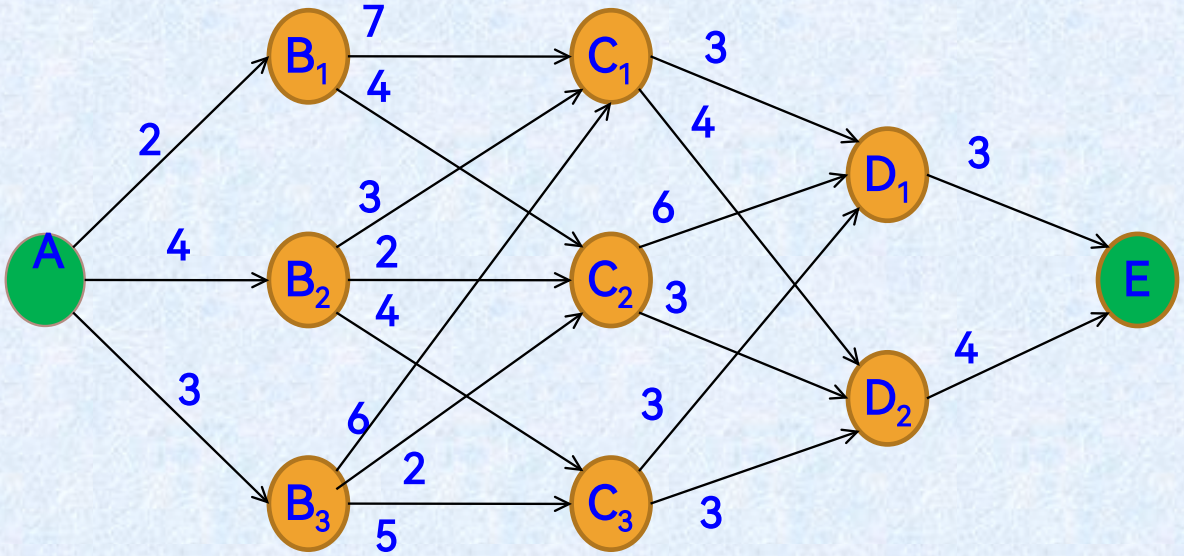


5.1.2 动态规划的原理

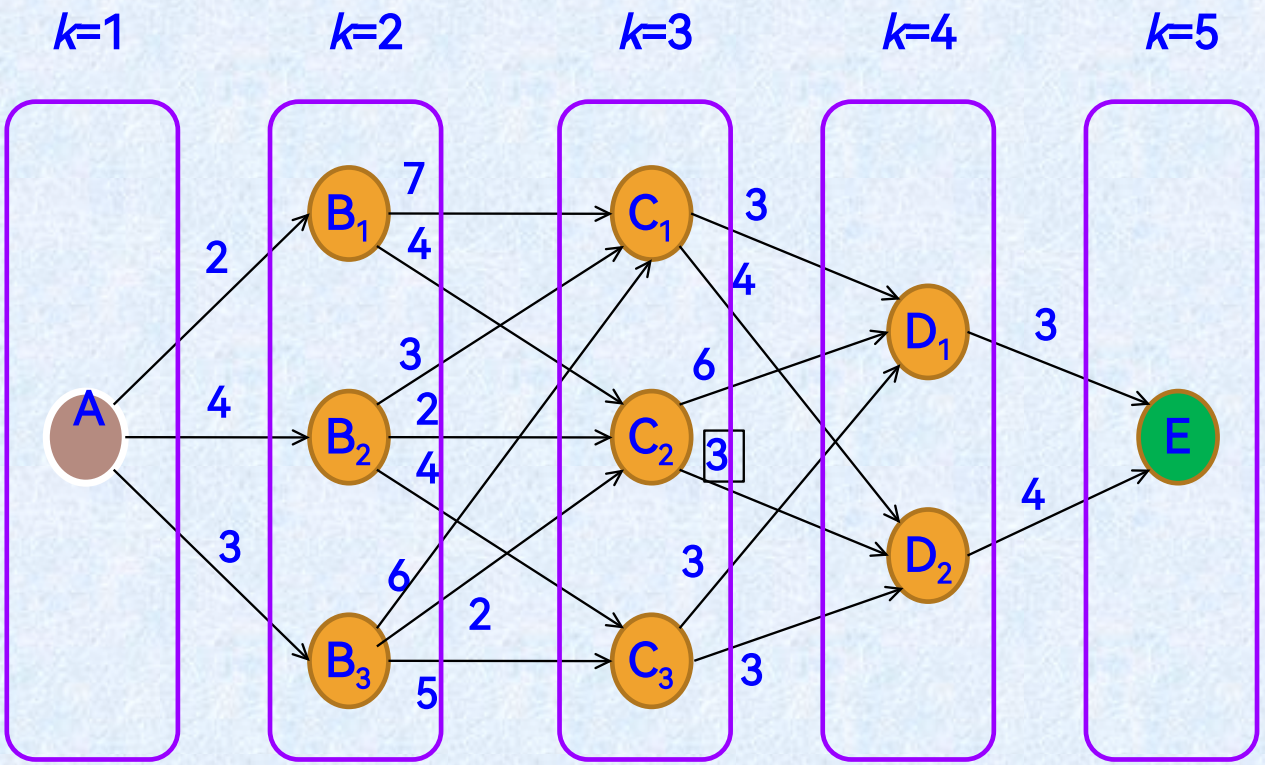
动态规划是一种解决多阶段决策问题的优化方法，把多阶段过程转化为一系列单阶段问题，利用各阶段之间的关系，逐个求解。

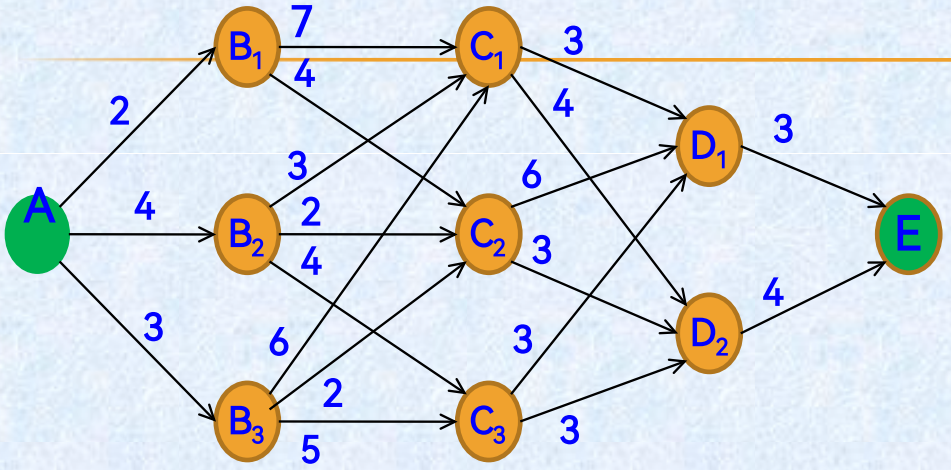
1. 动态规划的相关概念

示例：在*A*处有一水库，现需要从*A*点铺设一条管道到*E*点，边上的数字表示与其相连的两个地点之间所需修建的管道长度，用*c*数组表示，例如*c*(*A*，B_1)=2。现要找出一条从*A*到*E*的修建线路，使得所需修建的管道长度最短。



在从$A\sim E$的过程中，依据按位置所做的决策的次数及所做决策的先后次序，将问题分为5个阶段，阶段变量用于表示各阶段，这里阶段变量k为1～5，图中第5阶段是虚拟的一个边界阶段。





设最优指标函数$f(s)$表示状态s到终点E的最短路径长度，用k表示阶段，则对应的状态转移方程如下：

$$f_5(E)=0$$

$$f_k(s_k)=\min_{x_k\in D_k(s_k)}\{c(s_k,x_k)+f_{k+1}(s_{k+1})\}$$

符号示例

$S_2=\{B_1, B_2, B_3\}$

s_2取S_2的元素

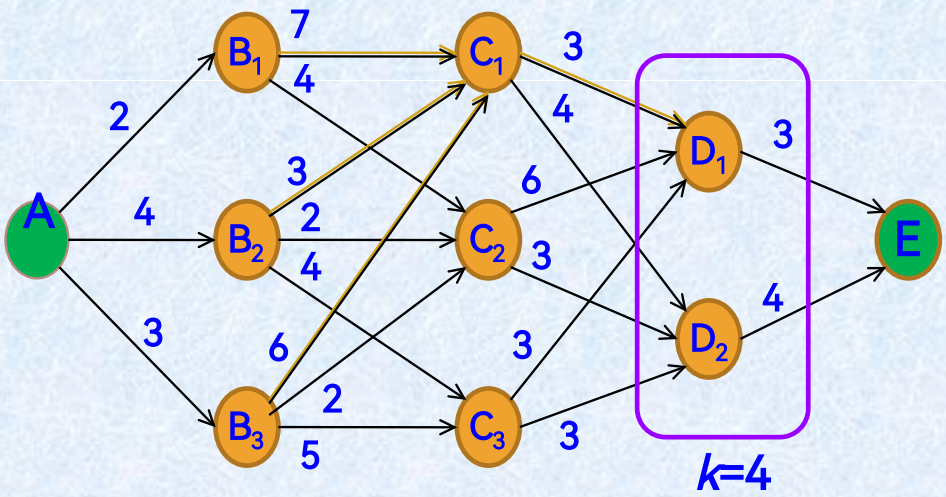
$D_2(B_1)=\{C_1, C_2\}$

x_2取$D_2(B_1)$的元素

2. 动态规划问题的解法

- 逆序解法
- 顺序解法

（1）动态规划问题的逆序解法



求解E→A的过程（用next表示路径上一个顶点的后继顶点）：

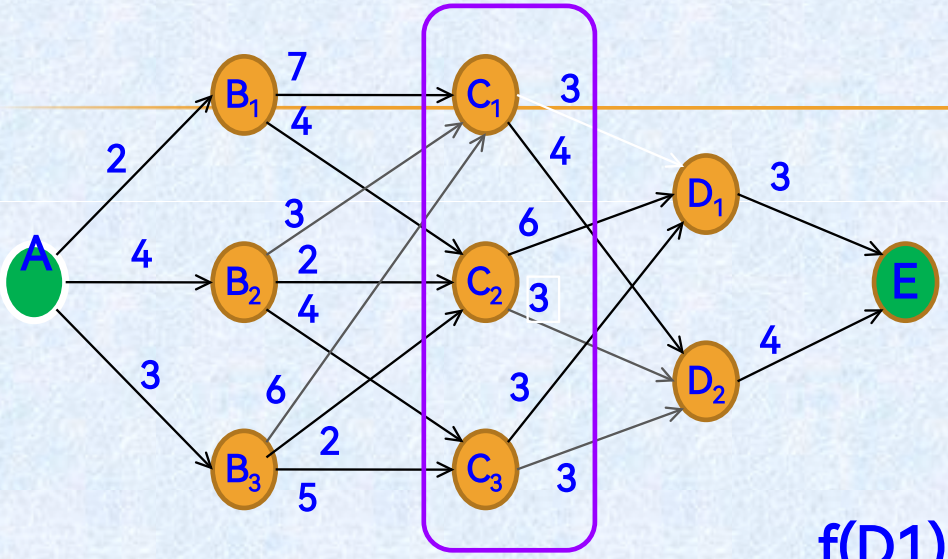
① 第5阶段

$f(E)=0$

② 第4阶段

$f(D_1)=MIN(c(D_1, E)+f(E))=3$，　$next(D_1)=E$

$f(D_2)=MIN(c(D_2, E)+f(E))=4$，　$next(D_2)=E$



k=3

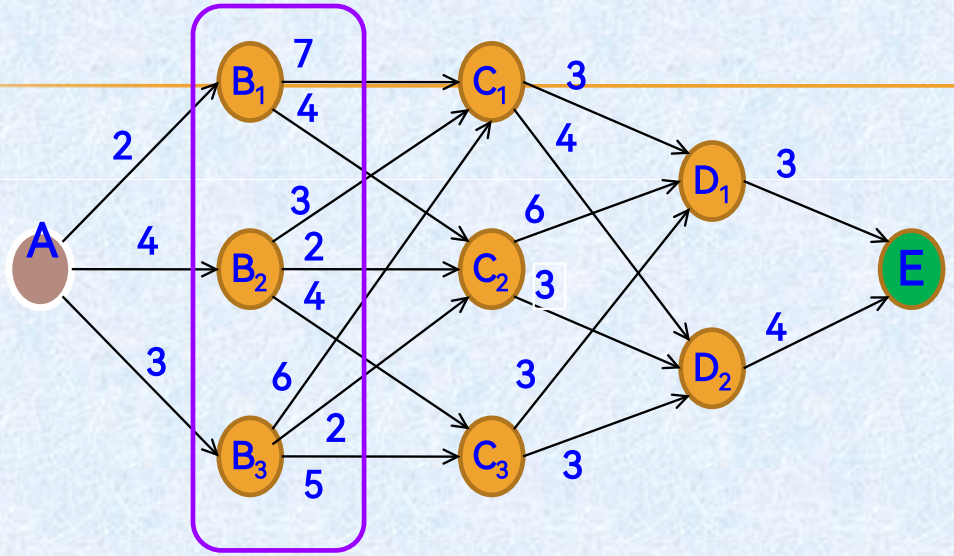
f(D1)=3
f(D2)=4

③ 第3阶段

$$f(C_1)=\text{MIN}\begin{pmatrix}c(C_1,D_1)+f(D_1)=6\\ c(C_1,D_2)+f(D_2)=8\end{pmatrix}=6,\quad \text{next}(C_1)=D_1$$

$$f(C_2)=\text{MIN}\begin{pmatrix}c(C_2,D_1)+f(D_1)=9\\ c(C_2,D_2)+f(D_2)=7\end{pmatrix}=7,\quad \text{next}(C_2)=D_2$$

$$f(C_3)=\text{MIN}\begin{pmatrix}c(C_3,D_1)+f(D_1)=6\\ c(C_3,D_2)+f(D_2)=7\end{pmatrix}=6,\quad \text{next}(C_3)=D_1$$



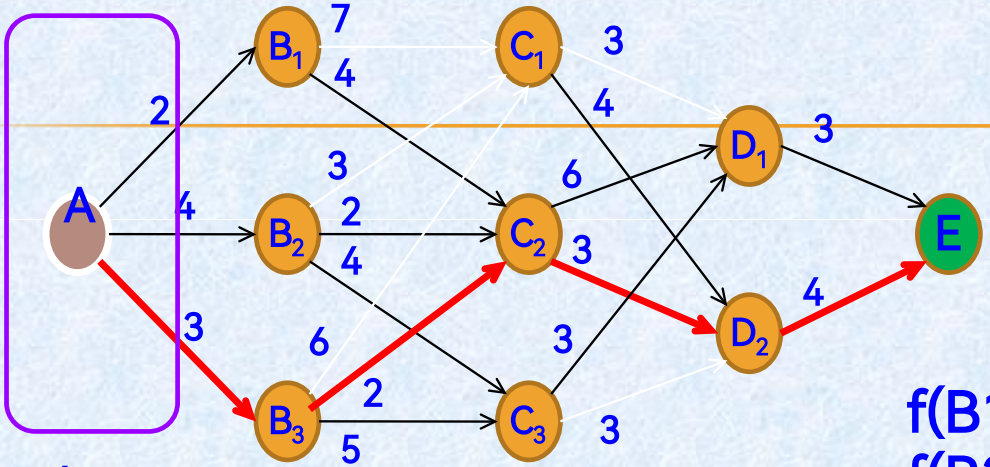
f(C1)=6
f(C2)=7
f(C3)=6

④ 第2阶段　k=2

$$f(B_1)=\text{MIN}\begin{pmatrix} c(B_1,C_1)+f(C_1)=13 \\ c(B_1,C_2)+f(C_2)=11 \\ c(B_1,C_3)+f(C_3)=\infty \end{pmatrix}=11\ ,\ \text{next}(B_1)=C_2$$

$$f(B_2)=\text{MIN}\begin{pmatrix} c(B_2,C_1)+f(C_1)=9 \\ c(B_2,C_2)+f(C_2)=9 \\ c(B_2,C_3)+f(C_3)=10 \end{pmatrix}=9\ ,\ \text{next}(B_2)=C_1$$

$$f(B_3)=\text{MIN}\begin{pmatrix} c(B_3,C_1)+f(C_1)=12 \\ c(B_3,C_2)+f(C_2)=9 \\ c(B_3,C_3)+f(C_3)=11 \end{pmatrix}=9\ ,\ \text{next}(B_3)=C_2$$



k=1

f(B1)=11
f(B2)=9
f(B3)=9

⑤ 第1阶段

$$f(\mathrm{A}) = \mathrm{MIN}\begin{pmatrix} c(\mathrm{A},\mathrm{B}_1)+f(\mathrm{B}_1)=13 \\ c(\mathrm{A},\mathrm{B}_2)+f(\mathrm{B}_2)=13 \\ c(\mathrm{A},\mathrm{B}_3)+f(\mathrm{B}_3)=12 \end{pmatrix} = 12 \quad ， \quad \mathrm{next(A)=B_3}$$

由*f*(A)=12求出的最短路径长度为12

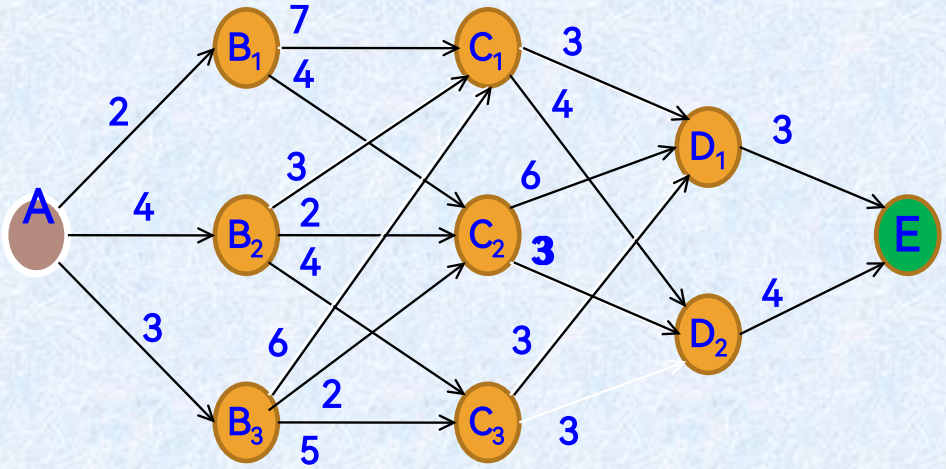
由next(A)=B_3，next(B_3)=C_2，next(C_2)=D_2，next(D_2)=E，推出最短路径为A→B_3→C_2→D_2→E。

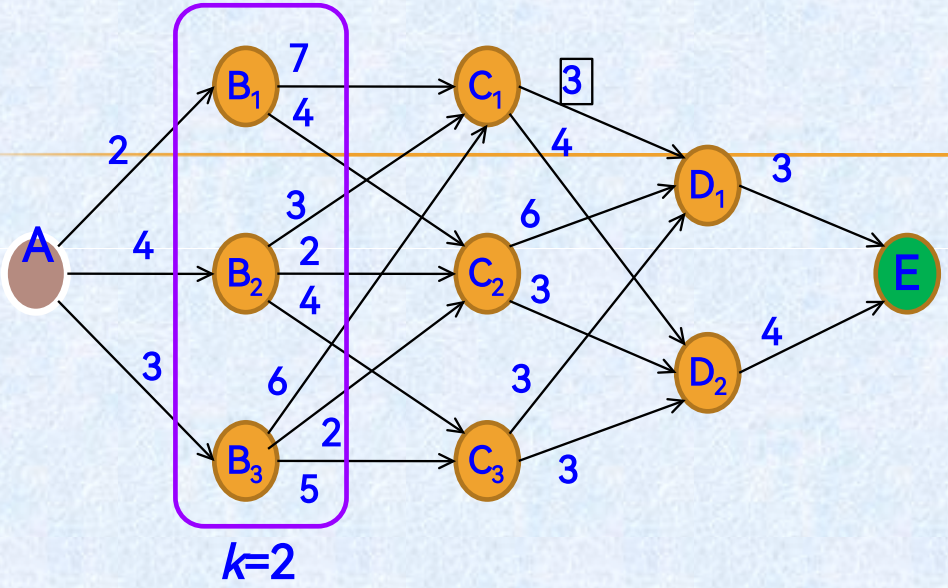
（2）动态规划问题的顺序解法

A→E对应的的状态转移方程如下：

$$f(A) = 0$$

$$f(s) = \underset{\text{存在}<t,s>\text{的有向边}}{\text{MIN}} \{ f(t) + c(t, s) \}$$





用pre表示路径上一个顶点的前驱顶点，其求解$A \rightarrow E$的过程：

① 第1阶段

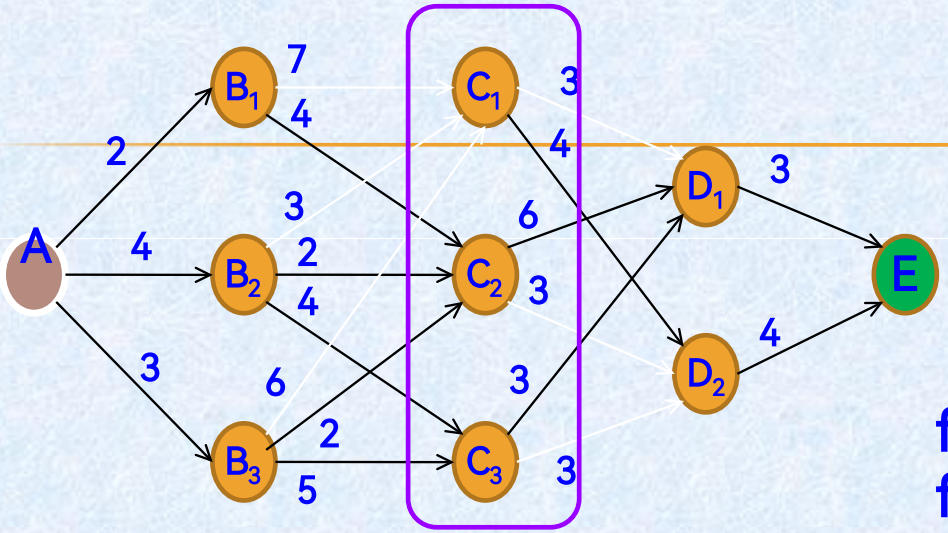
$f(A)=0$

② 第2阶段

$f(B_1)=MIN(f(A)+c(A, B_1))=2$，　$pre(B_1)=A$

$f(B_2)=MIN(f(A)+c(A, B_2))=4$，　$pre(B_2)=A$

$f(B_3)=MIN(f(A)+c(A, B_3))=3$，　$pre(B_3)=A$



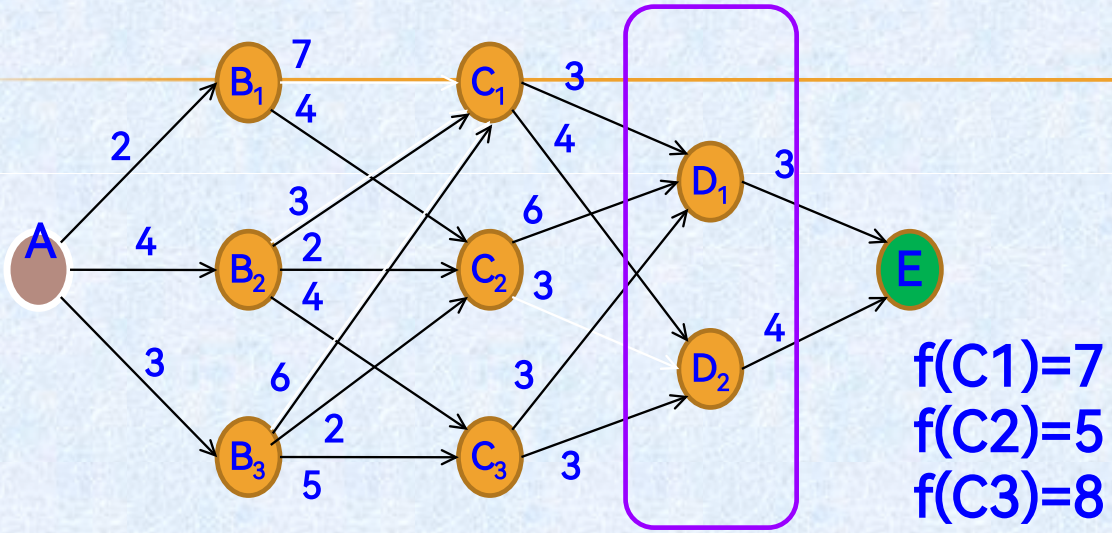
f(B1)=2
f(B2)=4
f(B3)=3

③ 第3阶段

$$f(C_1)=\mathrm{MIN}\begin{pmatrix} f(B_1)+c(B_1,C_1)=9 \\ f(B_2)+c(B_2,C_1)=7 \\ f(B_3)+c(B_3,C_1)=9 \end{pmatrix}=7,\quad \mathrm{pre}(C_1)=B_2$$

$$f(C_2)=\mathrm{MIN}\begin{pmatrix} f(B_1)+c(B_1,C_2)=6 \\ f(B_2)+c(B_2,C_2)=6 \\ f(B_3)+c(B_3,C_2)=5 \end{pmatrix}=5,\quad \mathrm{pre}(C_2)=B_3$$

$$f(C_3)=\mathrm{MIN}\begin{pmatrix} f(B_1)+c(B_1,C_3)=\infty \\ f(B_2)+c(B_2,C_3)=8 \\ f(B_3)+c(B_3,C_3)=8 \end{pmatrix}=8,\quad \mathrm{pre}(C_3)=B_2$$



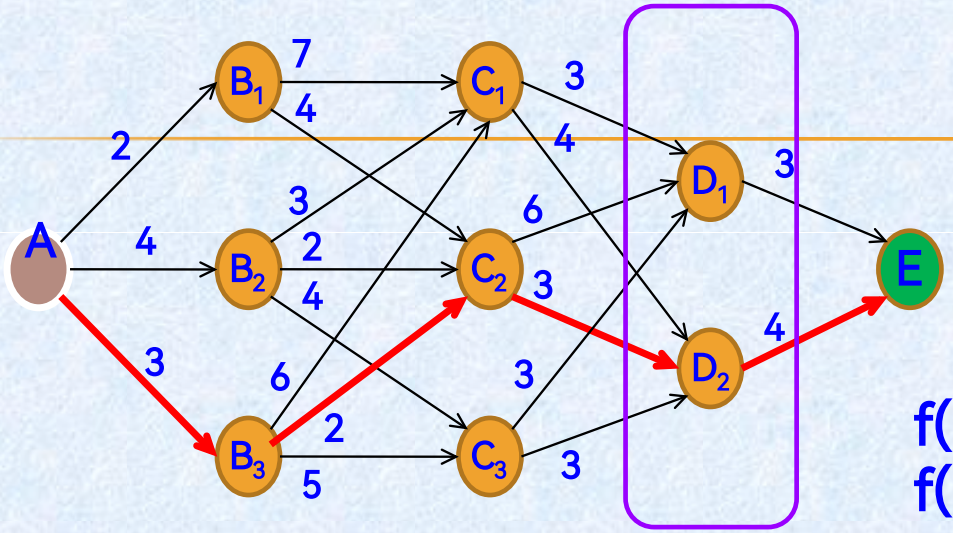
f(C1)=7
f(C2)=5
f(C3)=8

k=4

④ 第4阶段

$$f(D_1)=\mathrm{MIN}\begin{pmatrix} f(C_1)+c(C_1,D_1)=10 \\ f(C_2)+c(C_2,D_1)=11 \\ f(C_3)+c(C_3,D_1)=11 \end{pmatrix}=10,\quad \mathrm{pre}(D_1)=C_1$$

$$f(D_2)=\mathrm{MIN}\begin{pmatrix} f(C_1)+c(C_1,D_2)=11 \\ f(C_2)+c(C_2,D_2)=\ \ 8 \\ f(C_3)+c(C_3,D_2)=11 \end{pmatrix}=8,\quad \mathrm{pre}(D_2)=C_2$$



f(D1)=10
f(D2)=8

k=4

⑤ 第5阶段

$$f(\mathrm{E}) = \mathrm{MIN}\begin{pmatrix} f(\mathrm{D}_1) + \mathrm{c}(\mathrm{D}_1,\mathrm{E}) = 13 \\ f(\mathrm{D}_2) + \mathrm{c}(\mathrm{D}_2,\mathrm{e}) = 12 \end{pmatrix} = 12,\ \mathrm{pre(E)=D_2}$$

由*f*(E)=12求出的最短路径长度为12

由pre(E)=D_2，pre(D_2)=C_2，pre(C_2)=B_3，pre(B_3)=A，

推出最短路径为A→B_3→C_2→D_2→E。

5.1.3 动态规划求解的基本步骤

能采用动态规划求解的问题的一般要具有3个性质：

- 最优性原理：如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构，即满足最优性原理。
- 无后效性：即某阶段状态一旦确定，就不受这个状态以后决策的影响。也就是说，某状态以后的过程不会影响以前的状态，只与当前状态有关。
- 有重叠子问题：即子问题之间是不独立的，一个子问题在下一阶段决策中可能被多次使用到。（该性质并不是动态规划适用的必要条件，但是如果没有这条性质，动态规划算法同其他算法相比就不具备优势）。

实际应用中简化的步骤：

① 分析最优解的性质，并刻画其结构特征。

② 递归的定义最优解。

③ 以自底向上或自顶向下的记忆化方式计算出最优值。

④ 根据计算最优值时得到的信息，构造问题的最优解。

5.1.4 动态规划与其他方法的比较

动态规划的基本思想与分治法类似，也是将待求解的问题分解为若干个子问题（阶段），按顺序求解子阶段，前一子问题的解，为后一子问题的求解提供了有用的信息。

在求解任一子问题时，列出各种可能的局部解，通过决策保留那些有可能达到最优的局部解，丢弃其他局部解。依次解决各子问题，最后一个子问题就是初始问题的解。

动态规划方法又和贪心法有些相似，在动态规划中，可将一个问题的解决方案视为一系列决策的结果。

不同的是，在贪心法中，每采用一次贪心准则便做出一个不可回溯的决策，还要考察每个最优决策序列中是否包含一个最优子序列。

5.2 求解三角形最小路径问题

【问题描述】给定高度为*n*的一个整数三角形，找出从顶部到底部的最小路径和，只能向下移动相邻的结点。首先输入*n*，接下来的1 ~ *n*行，第*i*行输入*i*个整数，输出分为2行，第一行为最小路径，第2行为最小路径和。

例如，下图是一个*n*=4的三角形，输出的路径是2 3 5 3，最小路径是13。

【问题求解】将三角形采用二维数组*a*存放，前面的三角形对应的二维数组如下：

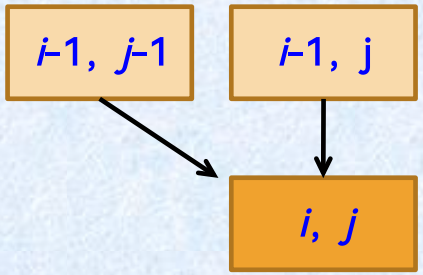
2

3　4

6　5　7

8　3　9　2

从顶部到底部查找最小路径，那么结点（*i*，*j*）的前驱结点只有（*i*-1，*j*-1）和（*i*-1，*j*）两个：



2			
3	4		
6	5	7	
8	3	9	2

a□□

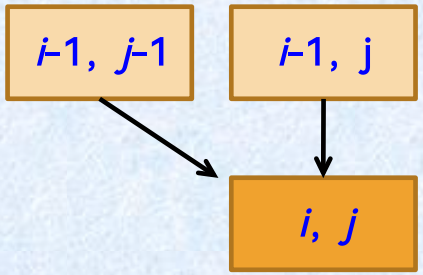
2			
5	6		
11	10	13	
19	13	19	15

dp□□

0			
0	0		
0	0	1	
0	1	1	2

pre□□

从顶部到底部查找最小路径，那么结点（*i*，*j*）的前驱结点只有（*i*-1，*j*-1）和（*i*-1，*j*）两个:



dp[*i*][*j*]表示从顶部*a*[0][0]查找到（*i*，*j*）结点时的最小路径和。显然这里有两个边界，即第1列和对角线，达到它们中结点的路径只有一条而不是常规的两条。所以状态转移方程如下：

dp[0][0]=*a*[0][0]　　顶部边界

dp[*i*][0]=dp[*i*-1][0]+*a*[*i*][0]　　考虑第1列的边界，1≤*i*≤*n*-1

dp[*i*][*i*]=dp[*i*-1][*i*-1]+*a*[*i*][*i*]　　考虑对角线的边界，1≤*i*≤*n*-1

dp[*i*][*j*]=min(dp[*i*-1][*j*-1]，dp[*i*-1][*j*])+*a*[*i*][*j*]　*i*>1的其他有两条达到路径的结点

最后求出的最小路径和ans=min(dp[*n*-1][*j*])，以及对应的列号*k*。

由dp求一条路径?

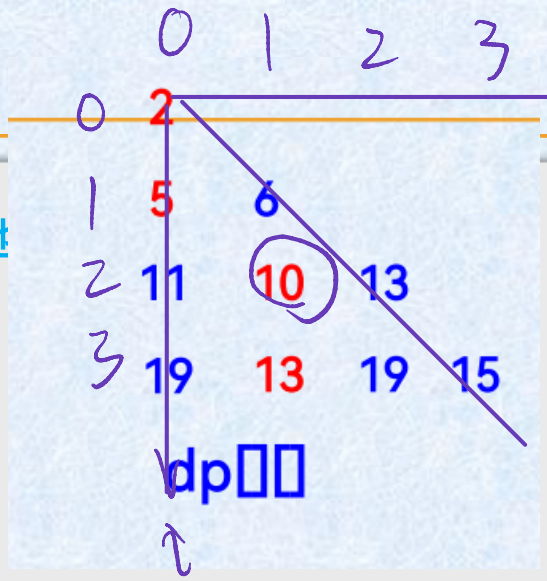
用pre[*i*][*j*]表示查找到（*i*，*j*）结点时最小路径上的前驱结点，由于前驱结点只有两个，即（*i*-1，*j*-1）和（*i*-1，*j*），用pre[*i*][*j*]记录前驱结点的列号即可。

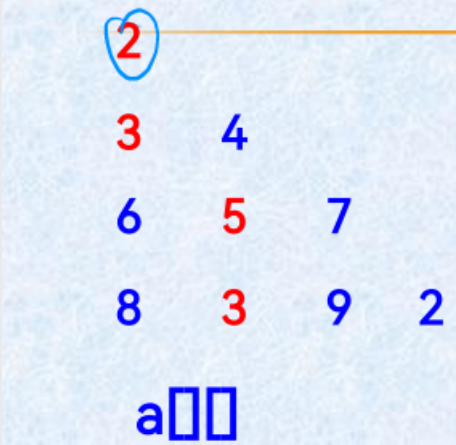
在求出ans后，通过pre[*n*-1][*k*]反推求出反向路径，最后正向输出该路径。

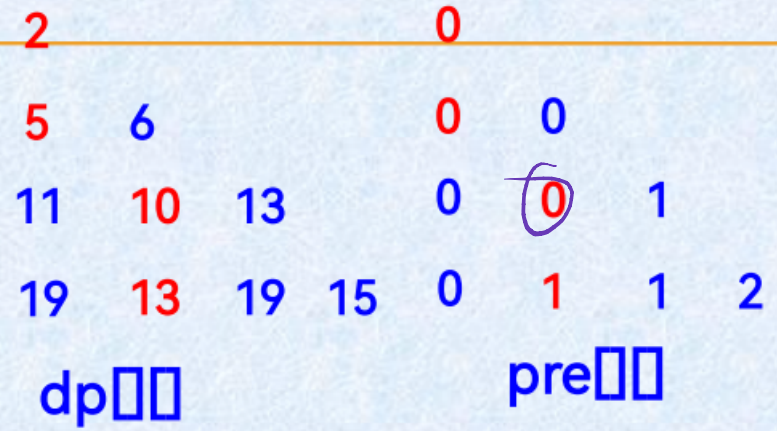
```
//问题表示
int a[MAXN][MAXN];
int n;
//求解结果表示
int ans=0;
int dp[MAXN][MAXN];
int pre[MAXN][MAXN];
```

```
int Search()                                    //求最小和路径ans
{  int i,  j;
   dp[0][0]=a[0][0];
   for(i=1;i<n;i++)                  //考虑第1列的边界
   {  dp[i][0]=dp[i-1][0]+a[i][0];
      pre[i][0]=0;
   }
   for (i=1;i<n;i++)                 //考虑对角线的边界
   {  dp[i][i]=a[i][i]+dp[i-1][i-1];
      pre[i][i]=i-1;
   }
```

```
for(i=2;i<n;i++)                    //考虑其他
{  for(j=1;j<i;j++)
   {  if(dp[i-1][j-1]<dp[i-1][j])
      {  pre[i][j]=j-1;
         dp[i][j]=a[i][j]+dp[i-1][j-1];
      }
      else
      {  pre[i][j]=j;
         dp[i][j]=a[i][j]+dp[i-1][j];
      }
   }
}
```







```
    ans=dp[n-1][0];
    int k=0;
    for (j=1;j<n;j++)                    //求出最小ans和对应的列号k
    {  if (ans>dp[n-1][j])
       {  ans=dp[n-1][j];
          k=j;
       }
    }
    return k;
}
```

```
int main()
{  int k;
   memset(pre，0，sizeof(pre));
   memset(dp，0，sizeof(dp));
   scanf("%d"，&n);                              //输入三角形的高度
   for (int i=0;i<n;i++)                         //输入三角形
      for (int j=0;j<=i;j++)
         scanf("%d"，&a[i][j]);
   k=Search();                                   //求最小路径和
   Disppath(k);                                  //输出正向路径
   printf("%d\n"，ans);                          //输出最小路径和
   return 0;
}
```

【算法分析】Search()算法中有i从2到$n-1$、j从1到$i-1$的两重循环，容易求出时间复杂度为$O(n^2)$。

5.3 求解最长公共子序列问题

【问题描述】字符序列的子序列是指从给定字符序列中随意地（不一定连续）去掉若干个字符（可能一个也不去掉）后所形成的字符序列。

令给定的字符序列$X=(x_0, x_1, \ldots, x_{m-1})$，序列$Y=(y_0, y_1, \ldots, y_{k-1})$是$X$的子序列，存在$X$的一个严格递增下标序列$(i_0, i_1, \ldots, i_{k-1})$，使得对所有的$j=0, 1, \ldots, k-1$，有 $=y_j$。

例如，$X=(a, b, c, b, d, a, b)$，$Y=(b, c, d, b)$是X的一个子序列。

给定两个序列A和B，称序列Z是A和B的公共子序列，是指Z同是A和B的子序列。该问题是求两序列A和B的最长公共子序列（LCS）。

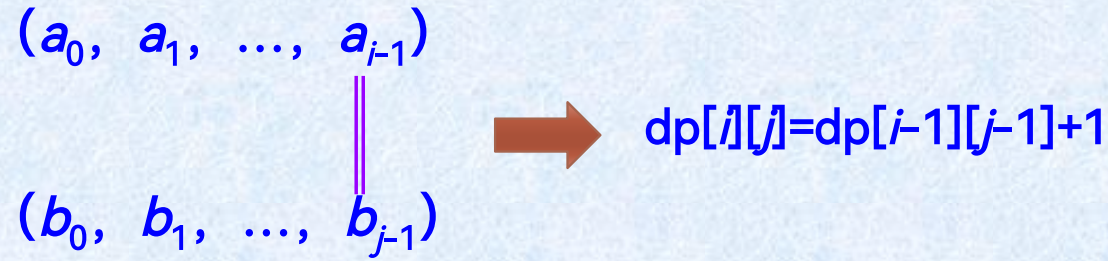
【问题求解】若设$A=(a_0, a_1, \ldots, a_{m-1})$，$B=(b_0, b_1, \ldots, b_{n-1})$，设$Z=(z_0, z_1, \ldots, z_{k-1})$为它们的最长公共子序列。不难证明有以下性质：

- 如果$a_{m-1}=b_{n-1}$，则$z_{k-1}=a_{m-1}=b_{n-1}$，且$(z_0, z_1, \ldots, z_{k-2})$是$(a_0, a_1, \ldots, a_{m-2})$和$(b_0, b_1, \ldots, b_{n-2})$的一个最长公共子序列。
- 如果$a_{m-1}\neq b_{n-1}$且$z_{k-1}\neq a_{m-1}$，则$(z_0, z_1, \ldots, z_{k-1})$是$(a_0, a_1, \ldots, a_{m-2})$和$(b_0, b_1, \ldots, b_{n-1})$的一个最长公共子序列。
- 如果$a_{m-1}\neq b_{n-1}$且$z_{k-1}\neq b_{n-1}$，则$(z_0, z_1, \ldots, z_{k-1})$是$(a_0, a_1, \ldots, a_{m-1})$和$(b_0, b_1, \ldots, b_{n-2})$的一个最长公共子序列。

定义二维动态规划数组dp，其中dp[*i*][*j*]为子序列（a_0，a_1，…，a_{i-1}）和（b_0，b_1，…，b_{j-1}）的最长公共子序列的长度。

每考虑一个字符*a*[*i*]或*b*[*j*]都为动态规划的一个阶段（共经历约*m*×*n*个阶段）。

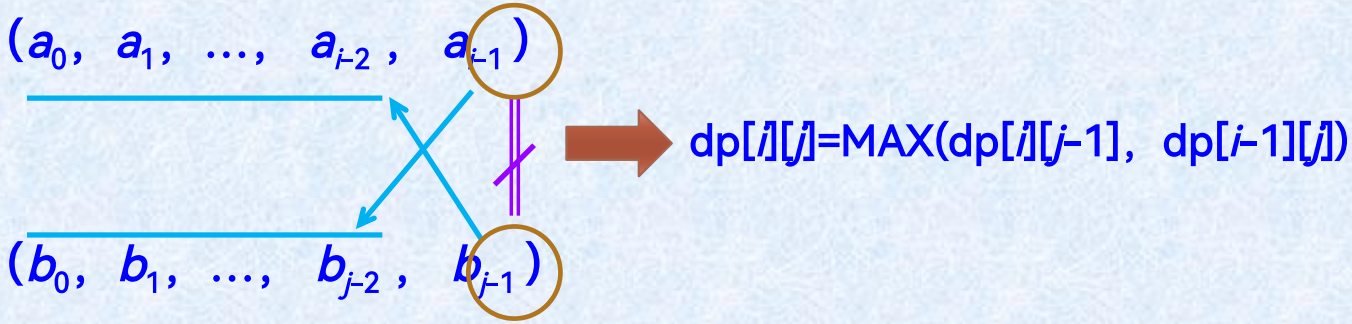
情况1：*a*[*i*-1]=*b*[*j*-1]



定义二维动态规划数组dp，其中dp[*i*][*j*]为子序列（a_0，a_1，…，a_{i-1}）和（b_0，b_1，…，b_{j-1}）的最长公共子序列的长度。

每考虑一个字符*a*[*i*]或*b*[*j*]都为动态规划的一个阶段（共经历约*m*×*n*个阶段）。

情况2：*a*[*i*-1]≠*b*[*j*-1]



dp[*i*][*j*]为子序列（a_0，a_1，…，a_{i-1}）和（b_0，b_1，…，b_{j-1}）的最长公共子序列的长度。

对应的状态转移方程如下：

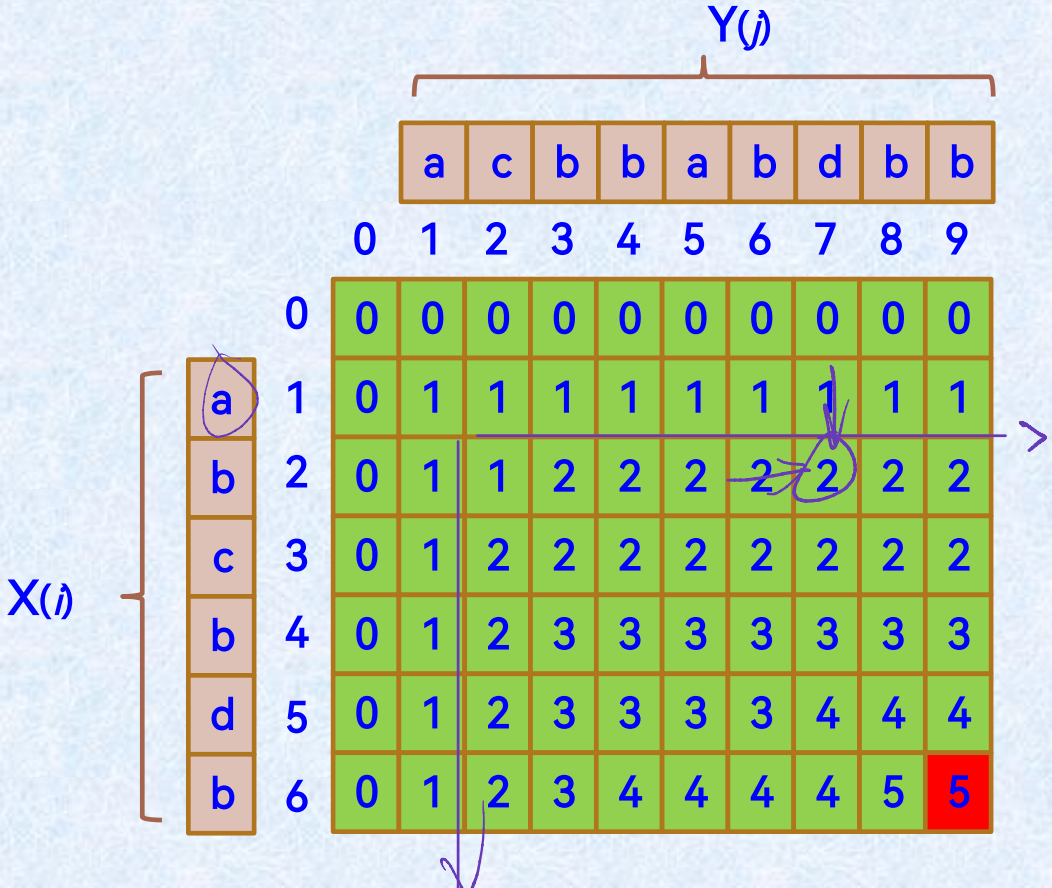
dp[*i*][*j*]=0	*i*=0或*j*=0—边界条件
dp[*i*][*j*]=dp[*i*-1][*j*-1]+1	*a*[*i*-1]=*b*[*j*-1]
dp[*i*][*j*]=MAX(dp[*i*][*j*-1]，dp[*i*-1][*j*])	*a*[*i*-1]≠*b*[*j*-1]

显然，dp[*m*][*n*]为最终结果。

dp[*i*][*j*]=0　*i*=0或*j*=0—边界条件

dp[*i*][*j*]=dp[*i*-1][*j*-1]+1　　*a*[*i*-1]=*b*[*j*-1]

dp[*i*][*j*]=MAX(dp[*i*][*j*-1]，dp[*i*-1][*j*])　　*a*[*i*-1]≠*b*[*j*-1]



for (i=>m)

for (j=>n)

dp

dp[i][j] = max{ dp[i-1][j], dp[i][j-1] }

那么如何由dp求出一个LCS呢?

dp[*i*][*j*]=0	*i*=0或*j*=0—边界条件
dp[*i*][*j*]=dp[*i*-1][*j*-1]+1	*a*[*i*-1]=*b*[*j*-1]
dp[*i*][*j*]=MAX(dp[*i*][*j*-1]，dp[*i*-1][*j*])	*a*[*i*-1]≠*b*[*j*-1]

❶当dp[*i*][*j*] ≠ dp[*i*][*j*-1]（左边）并且dp[*i*][*j*] ≠ dp[*i*-1][*j*]（上方）值时:

a[*i*-1]=*b*[*j*-1] ⇒ 它是LCS的一个字符

将*a*[*i*-1]添加到LCS中。

❷ 其他情况时：*a*[*i*-1]或者*b*[*j*-1]是LCS的一个字符!

总 结:

用vector<char>字符向量subs存放一个LCS，*k*=dp[*m*][*n*]（LCS的字符个数），从*k*到1循环求出subs中的*k*个字符:

2

（1）如果dp[*i*][*j*]==dp[*i*-1][*j*]（上方），说明*a*[*i*-1]或*b*[*j*-1]不是LCS中的字符 □ *i*--;

（2）如果dp[*i*][*j*]==dp[*i*][*j*-1]（左边），说明*a*[*i*-1]或*b*[*j*-1]不是LCS中的字符 □ *j*--;

1

（3）其他情况，说明*a*[*i*-1]或者*b*[*j*-1]是LCS的一个字符!

□ *i*--; *j*--; *k*--;表示求的字符减少1个

例如，X=（a，b，c，b，d，b），m=6，Y=（a，c，b，b，a，b，d，b，b），n=9。

（1）求出dp

（2）dp[6][9]=5开始

那么如何由dp求出LCS呢？例如，$X=(a, b, c, b, d, b)$，$m=6$，$Y=(a, c, b, b, a, b, d, b, b)$，$n=9$。

（1）求出dp

（2）dp[6][9]=5开始

LCS: c b d b

那么如何由dp求出LCS呢？例如，X=（a，b，c，b，d，b），m=6，Y=（a，c，b，b，a，b，d，b，b），n=9。

（1）求出dp

（2）dp[6][9]=5开始

LCS: a c b d b

i=2，j=1

与上方相等，i--

i=1，j=1

与左、上方不等，i--，j--

i=0，j=0

结果LCS="acbdb"

```
#define MAX 51                          //序列中最多的字符个数
//问题表示
int m，n;
string a，b;                            //求解结果表示
int dp[MAX][MAX];                       //动态规划数组
vector<char> subs;                      //存放LCS
```

0->j-1

```
void LCSlength()                          //求dp
{  int i，j;
   for (i=0;i<=m;i++)                                  //将dp[i][0]置为0，边界条件
      dp[i][0]=0;
   for (j=0;j<=n;j++)                                  //将dp[0][j]置为0，边界条件
      dp[0][j]=0;
   for (i=1;i<=m;i++)
      for (j=1;j<=n;j++)            //两重for循环处理a、b的所有字符
      {  if (a[i-1]==b[j-1])                 //情况(1)
            dp[i][j]=dp[i-1][j-1]+1;
         else                                          //情况(2)
            dp[i][j]=max(dp[i][j-1]，dp[i-1][j]);
      }
}
```

```
void Buildsubs()                    //由dp构造从subs
{  int k=dp[m][n];                  //k为a和b的最长公共子序列长度
   int i=m;
   int j=n;
   while (k>0)                                  //在subs中放入最长公共子序列(反向)
    if (dp[i][j]==dp[i-1][j])
            i--;
    else if (dp[i][j]==dp[i][j-1])
            j--;
    else                            //与上方、左边元素值均不相等
    {       subs.push_back(a[i-1]);  //subs中添加a[i-1]
            i--; j--; k--;
    }
}
```

【算法分析】 LCSlength算法中使用了两重循环，所以对于长度分别为m和n的序列，求其最长公共子序列的时间复杂度为$O(m\times n)$。空间复杂度为$O(m\times n)$。

5.4 求解最长递增子序列问题

【问题描述】给定一个无序的整数序列$a[0..n-1]$，求其中最长递增子序列的长度。

例如，$a[]=\{2，1，5，3，6，4，8，9，7\}$，$n=9$，其最长递增子序列为{1，3，4，8，9}，结果为5。

【问题求解】设计动态规划数组为一维数组dp，dp[*i*]表示*a*[0..*i*]中以*a*[*i*]结尾的最长递增子序列的长度。对应的状态转移方程如下：

dp[*i*]=1　*0≤i≤n-1*

dp[*i*]=max(dp[*i*]，dp[*j*]+1) *若a[i]>a[j]，0≤i≤n-1，0≤j≤i-1*

求出dp后，其中最大元素即为所求。

dp[*i*]=1 *0≤i≤n-1*

dp[*i*]=max(dp[*i*]，dp[*j*]+1) *若a[i]>a[j]，0≤i≤n-1，0≤j≤i-1*

i	0	1	2	3	4	5	6	7	8
a[i]	2	1	5	3	6	4	8	9	7
dp[i]	1	1	2	2	3	3	4	5	4

```
//问题表示
int a[]={2，1，5，3，6，4，8，9，7};
int n=sizeof(a)/sizeof(a[0]);
//求解结果表示
int ans=0;
int dp[MAX];

void solve(int a[]，int n)
{  int i，j;
   for(i=0;i<n;i++)
   {  dp[i]=1;
      for(j=0;j<i;j++)
      {  if (a[i]>a[j])
            dp[i]=max(dp[i]，dp[j]+1);
      }
   }
   ans=dp[0];
   for(i=1;i<n;i++)
     ans=max(ans，dp[i]);
}
```

【算法分析】solve()算法中含两重循环，时间复杂度为$O(n^2)$。

5.5 求解0/1背包问题

【问题描述】有n个重量分别为{w_1，w_2，…，w_n}的物品，它们的价值分别为{v_1，v_2，…，v_n}，给定一个容量为W的背包。

设计从这些物品中选取一部分物品放入该背包的方案，每个物品要么选中要么不选中，要求选中的物品不仅能够放到背包中，而且重量和为W具有最大的价值。

【问题求解】对于可行的背包装载方案，背包中物品的总重量不能超过背包的容量。

最优方案是指所装入的物品价值最高，即 $v_1*x_1+v_2*x_2+\ldots+v_n*x_n$（其中$x_i$取0或1，取1表示选取物品$i$）取得最大值。

在该问题中需要确定x_1、x_2、…、x_n的值。假设按i=1，2，…，n的次序来确定x_i的值，对应n次决策即n个阶段。

dp[i][r] =

设置二维动态规划数组dp，dp[*i*][*r*]表示背包剩余容量为*r*（1≤*r*≤*W*），已考虑物品1、2、…、*i*（1≤*i*≤*n*）时背包装入物品的最优价值。显然对应的状态转移方程如下：

dp[*i*][0]=0（背包不能装入任何物品，总价值为0）

边界条件dp[*i*][0]=0（1≤*i*≤*n*）—边界条件

dp[0][*r*]=0（没有任何物品可装入，总价值为0）

边界条件dp[0][*r*]=0（1≤*r*≤*W*）—边界条件

dp[*i*][*r*]=dp[*i*-1][*r*]当*r*<*w*[*i*]时，物品i放不下

dp[*i*][*r*]= MAX{dp[*i*-1][*r*]，dp[*i*-1][*r*-*w*[*i*]]+*v*[*i*]}

否则在不放入和放入物品*i*之间选最优解

这样， dp[*n*][*W*]便是0/1背包问题的最优解。

当dp数组计算出来后，推导出解向量x的过程十分简单，从dp[n][W]开始：

（1）若dp[i][r]≠dp[i-1][r]，若dp[i][r]=dp[i-1][r-w[i]]+v[i]，置x[i]=1，累计总价值maxv+=v[i]，递减剩余重量r=r-w[i]。

（2）若dp[i][r]=dp[i-1][r]，表示物品i放不下或者不放入物品i，置x[i]=0。

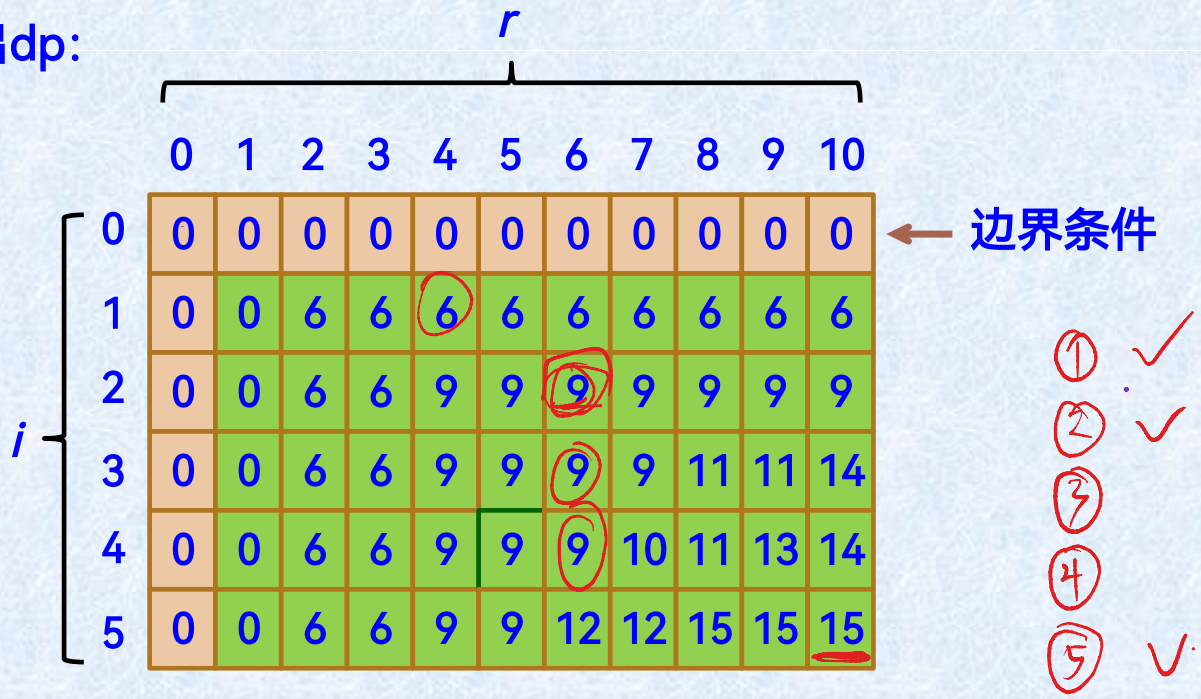
dp[i][r]=dp[i-1][r]当r<w[i]时，物品i放不下

dp[i][r]= MAX{dp[i-1][r]，dp[i-1][r-w[i]]+v[i]}

否则在不放入和放入物品i之间选最优解

例如，某0/1背包问题为，n=5，w={2，2，6，5，4}，v={6，3，5，4，6}（下标从1开始），W=10。

求出dp:



dp[i][r]=dp[i-1][r] 当r<w[i]时，物品i放不下

dp[i][r]= MAX{dp[i-1][r]，dp[i-1][r-w[i]]+v[i]}

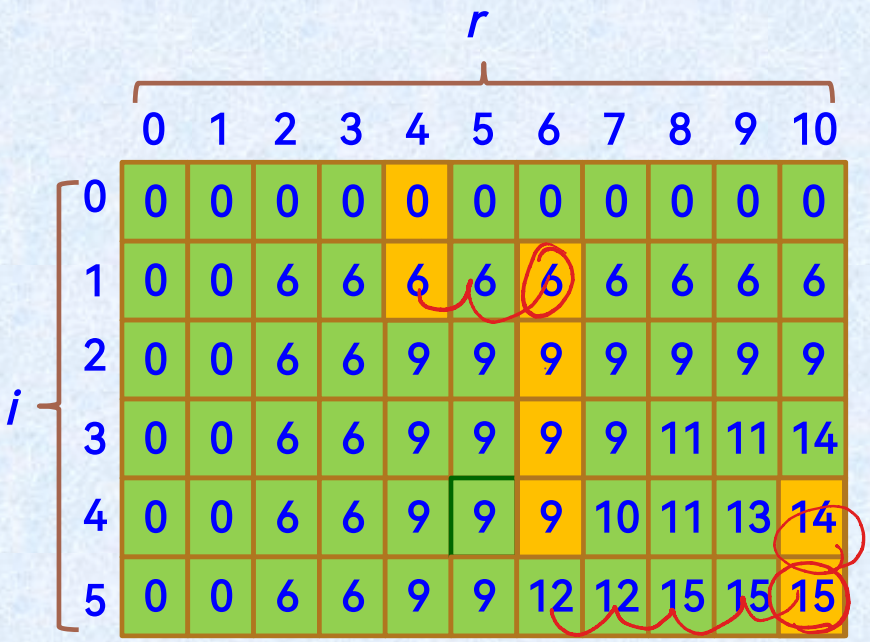
否则在不放入和放入物品i之间选最优解

回推求最优解的过程：

w={2，2，6，5，4}

i=5，r=W=10，从dp[5][10]开始

dp[5][10] ≠dp[4][10]
⇒ x[5]=1，r=r-w[5]=6



i=i-1=4，dp[4][6]=dp[3][6]
⇒ x[4]=0

i=i-1=3，dp[3][6]=dp[2][6]
⇒ x[3]=0

i=i-1=2，dp[2][6]≠dp[1][6]
⇒ x[2]=1，r=r-w[2]=4

i=i-1=1，dp[1][4]≠dp[0][4]
⇒ x[1]=1，r=r-w[1]=2

x=（1，1，0，0，1），
装入物品总重量为8，
总价值为15

```
//问题表示
int n=5，W=10;                          //5种物品，限制重量不超过10
int w[MAXN]={0，2，2，6，5，4}; //下标0不用
int v[MAXN]={0，6，3，5，4，6}; //下标0不用
//求解结果表示
int dp[MAXN][MAXW];
int x[MAXN];
int maxv;                               //存放最优解的总价值
```

```
void Knap()                          //动态规划法求0/1背包问题
{  int i，  r;
   for (i=0;i<=n;i++)                //置边界条件dp[i][0]=0
      dp[i][0]=0;
   for (r=0;r<=W;r++)                //置边界条件dp[0][r]=0
      dp[0][r]=0;

   for (i=1;i<=n;i++)
   {  for (r=1;r<=W;r++)
       if (r<w[i])
          dp[i][r]=dp[i-1][r];
       else
          dp[i][r]=max(dp[i-1][r]，dp[i-1][r-w[i]]+v[i]);
   }
}
```

```
void Buildx()                                   //回推求最优解
{  int i=n,  r=W;
   maxv=0;
   while (i>=0)                                 //判断每个物品
   {
      if (dp[i][r]!=dp[i-1][r])
      {  x[i]=1;                                //选取物品i
         maxv+=v[i];                            //累计总价值
         r=r-w[i];
      }
      else
        x[i]=0;                                 //不选取物品i
      i--;
   }
}
```

【算法分析】Knap()算法中含有两重for循环，所以时间复杂度为O($n \times W$)，空间复杂度为O($n \times W$)。

5.6 完全加括号的矩阵连乘积

- 完全加括号的矩阵连乘积可递归地定义为：
  （1）单个矩阵是完全加括号的；
  （2）矩阵连乘积A 是完全加括号的，则 A可表示为2个完全加括号的矩阵连乘积 B和C的乘积并加括号，即A=（BC）
- 设有四个矩阵 A, B, C, D 它们的维数分别是：

$A = 50 \times 10$ $B = 10 \times 40$ $C = 40 \times 30$ $D = 30 \times 5$

- 总共有五中完全加括号的方式

$(A((BC)D))$ $(A(B(CD)))$ $((AB)(CD))$

$(((AB)C)D)$ $((A(BC))D)$

16000, 10500, 36000, 87500, 34500

- 给定n个矩阵$\{A_1, A_2, \ldots, A_n\}$，其中$A_i$与$A_{i+1}$是可乘的，$i = 1,2,\ldots,n-1$。考察这n个矩阵的连乘积

$$A_1 A_2 \ldots A_n$$

- 由于矩阵乘法满足结合律，所以计算矩阵的连乘可以有许多不同的计算次序。这种计算次序可以用加括号的方式来确定。
- 若一个矩阵连乘积的计算次序完全确定，也就是说该连乘积已完全加括号，则可以依此次序反复调用2个矩阵相乘的标准算法计算出矩阵连乘积

给定n个矩阵{$A_1,A_2,\ldots,A_n$}，其中A_i与A_i+1是可乘的，i=1,2 ,…,n-1。如何确定计算矩阵连乘积的计算次序，使得依此次序计算矩阵连乘积需要的数乘次数最少。

- 穷举法：列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

  算法复杂度分析：
  对于n个矩阵的连乘积，设其不同的计算次序为P(n)。
  由于每种加括号方式都可以分解为两个子矩阵的加括号问题：
  $(A_1\ldots A_k)(A_k+1\ldots A_n)$可以得到关于P(n)的递推式如下：

$$P(n)=\begin{cases}1 & n=1\\ \sum_{k=1}^{n-1}P(k)P(n-k) & n>1\end{cases}$$

- 穷举法
- 动态规划

将矩阵连乘积 $A_iA_{i+1}\dots A_j$ 简记为A[i:j]，这里i≤j

考察计算A[i:j]的最优计算次序。设这个计算次序在矩阵Ak和Ak+1之间将矩阵链断开，i≤k<j，则其相应完全加括号方式为

$$(A_iA_{i+1}\dots A_k)(A_{k+1}A_{k+2}\dots A_j)$$

计算量：A[i:k]的计算量加上A[k+1:j]的计算量，再加上A[i:k]和A[k+1:j]相乘的计算量

分析最优解的结构

- 特征：计算A[i:j]的最优次序所包含的计算矩阵子链 A[i:k]和A[k+1:j]的次序也是最优的。
- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为最优子结构性质。问题的最优子结构性质是该问题可用动态规划算法求解的显著特征。

建立递归关系

- 设计算A[i:j]，1≤i≤j≤n，所需要的最少数乘次数m[i,j]，则原问题的最优值为m[1,n]
- 当i=j时，A[i:j]=Ai，因此，m[i,i]=0，i=1,2,…,n
- 当i<j时，

$$m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_k p_j$$

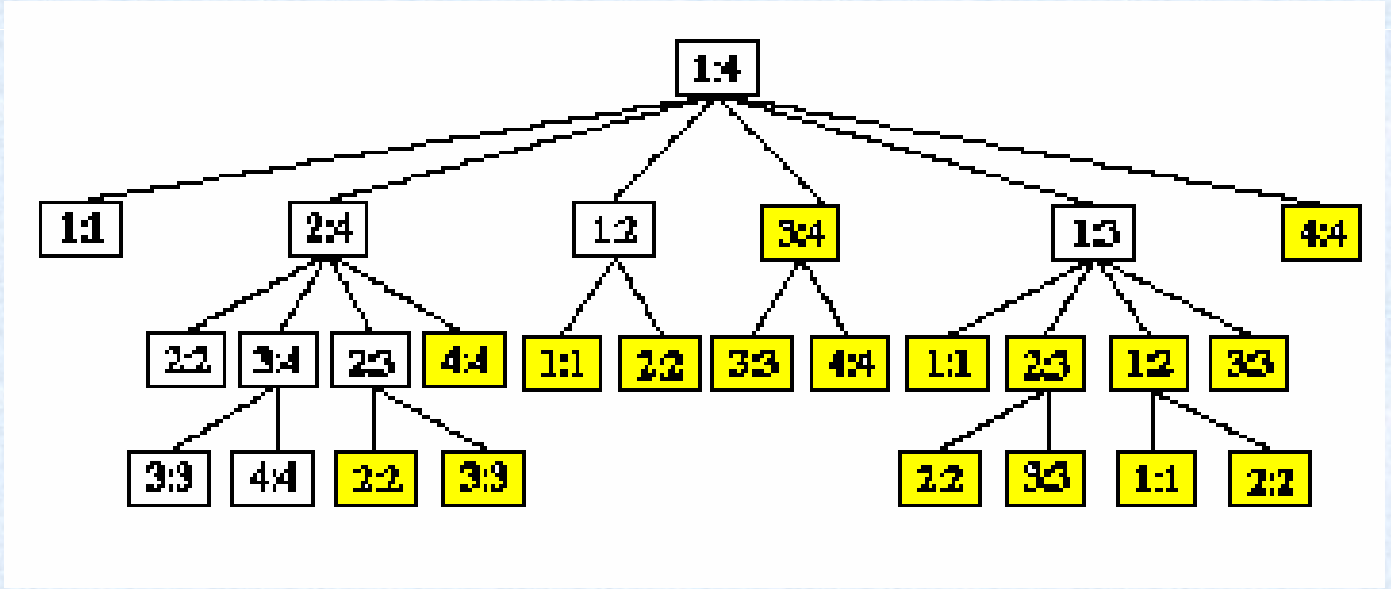
- 可以递归地定义m[i,j]为：

这里 A_i 的维数为 $p_{i-1} \times p_i$

$$m[i,j] = \begin{cases} 0 & i = j \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\} & i < j \end{cases}$$

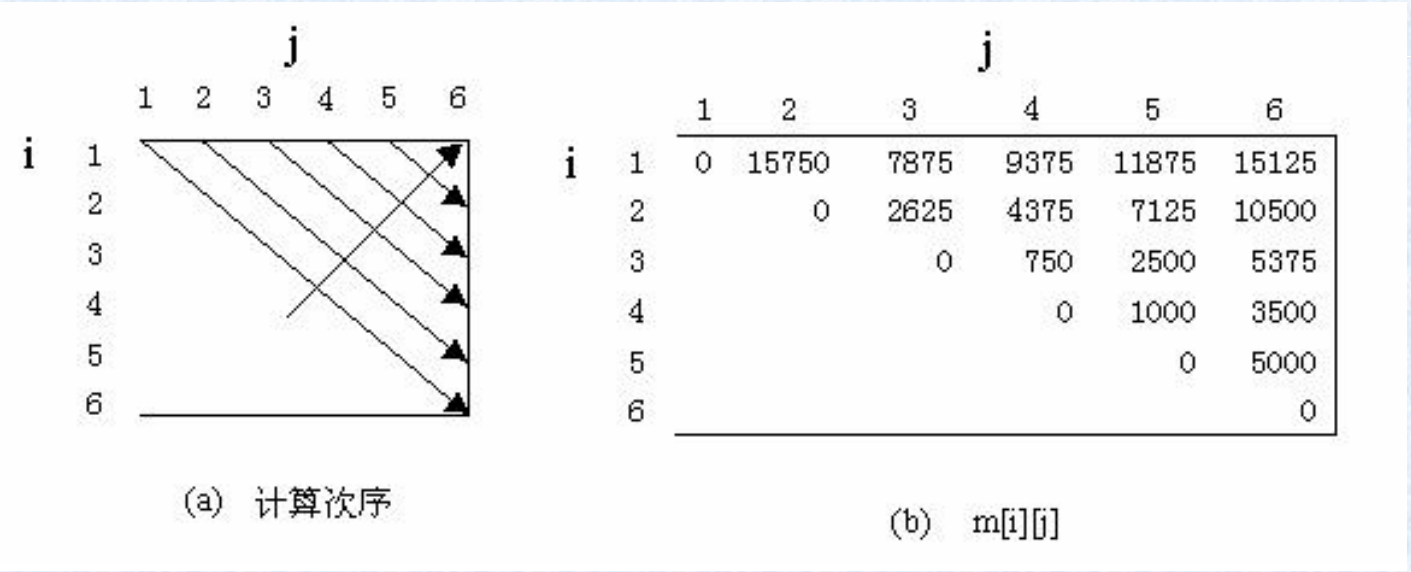
k的位置只有j-i 种可能

```
int matrixchain(int i,int j)
{ if(i==j) return 0;
  int u=matrixchain(i+1,j)+p[i-1]*p[i]*p[j];
  s[i][j]=i; //初始化断点位置为i
  for(int k=i+1;k<j;k++){
   int t;
   t=matrixchain(i,k)+matrixchain(k+1,j)+p[i-1]*p[k]*p[j];
   if(t<u){
      u=t;
      s[i][j]=k;}
    }//end for
   return u;
}
```

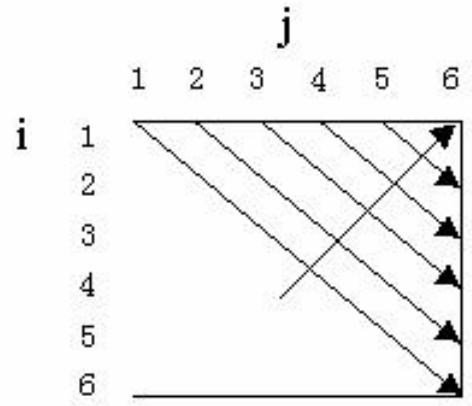

1:4
1:1
2:4
1:2
3:4
1:3
4:4
2:2
3:4
2:3
4:4
1:1
2:2
3:3
4:4
1:1
2:3
1:2
3:3
3:3
4:4
2:2
3:3
2:2
3:3
1:1
2:2

计算最优值

- 对于$1 \le i \le j \le n$不同的有序对(i,j)对应于不同的子问题。不同子问题的个数最多只有$\theta(n^2)$.
- 在递归计算时，许多子问题被重复计算多次。这也是该问题可用动态规划算法求解的又一显著特征。
- 用动态规划算法解此问题，可依据其递归式以自底向上的方式进行计算。在计算过程中，保存已解决的子问题答案。每个子问题只计算一次，而在后面需要时只要简单查一下，从而避免大量的重复计算，最终得到多项式时间的算法



A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25



(a) 计算次序

i \ j	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

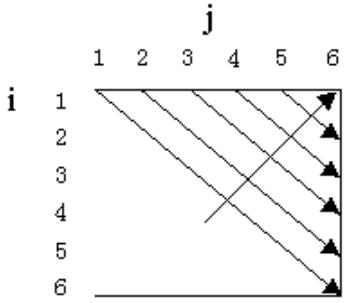
(b) m[i][j]

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$

```
public static void matrixChain(int [] p, int [][] m, int [][] s)
  {
    int n=p.length-1;
    for (int i = 1; i <= n; i++) m[i][i] = 0;
    for (int r = 2; r <= n; r++)  //2到n列
      for (int i = 1; i <= n - r+1; i++) { //1到n-r+1行
        int j=i+r-1;
        m[i][j] = m[i+1][j]+ p[i-1]*p[i]*p[j];
        s[i][j] = i;
```

```
for (int k = i+1; k < j; k++) {
        int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
        if (t < m[i][j]) {
          m[i][j] = t;
          s[i][j] = k;}
        }//end for k
      }//end for i
  }// end for
}// end function
```



(a) 计算次序

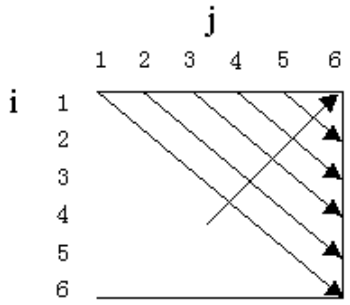
i \ j	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

(b) m[i][j]

i \ j	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

(c) s[i][j]

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25



(a) 计算次序

j

i	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

(b) m[i][j]

j

i	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

(c) s[i][j]

(A1(A2A3))(A4A5)A6)

算法复杂度分析：
算法matrixChain的主要计算量取决于算法中对r，i和k的3重循环。循环体内的计算量为O(1)，而3重循环的总次数为$O(n^3)$。因此算法的计算时间上界为$O(n^3)$。算法所占用的空间显然为$O(n^2)$。

5.7 动态规划算法的基本要素

一、最优子结构

- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为最优子结构性质。

- 在分析问题的最优子结构性质时，所用的方法具有普遍性：首先假设由问题的最优解导出的子问题的解不是最优的，然后再设法说明在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。

- 利用问题的最优子结构性质，以自底向上的方式递归地从子问题的最优解逐步构造出整个问题的最优解。最优子结构是问题能用动态规划算法求解的前提。

注意：同一个问题可以有多种方式刻划它的最优子结构，有些表示方法的求解速度更快（空间占用小，问题的维度低）

二、重叠子问题

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为子问题的重叠性质。
- 动态规划算法，对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。
- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。

三、备忘录方法

- 备忘录方法的控制结构与直接递归方法的控制结构相同，区别在于备忘录方法为每个解过的子问题建立了备忘录以备需要时查看，避免了相同子问题的重复求解。

```
private static int lookupChain(int i, int j)
  {
    if (m[i][j] > 0) return m[i][j];
    if (i == j) return 0;
    int u = lookupChain(i+1,j) + p[i-1]*p[i]*p[j];
    s[i][j] = i;
    for (int k = i+1; k < j; k++) {
     int t ;
     t= lookupChain(i,k) + lookupChain(k+1,j) + p[i-1]*p[k]*p[j];
       if (t < u) {
          u = t; s[i][j] = k;}
       }//end for
     m[i][j] = u;
     return u;
  }
```

5.8 求解资源分配问题

【问题描述】资源分配问题是将数量一定的一种或若干种资源（原材料、资金、设备或劳动力等），合理地分配给若干使用者，使总收益最大。

例如，某公司有3个商店A、B、C，拟将新招聘的5名员工分配给这3个商店，各商店得到新员工后，每年的赢利情况如下表所示，求分配给各商店各多少员工才能使公司的赢利最大?

员工数 / 商店	0人	1人	2人	3人	4人	5人
A	0	3	7	9	12	13
B	0	5	10	11	11	11
C	0	4	6	11	12	12

【问题求解】采用动态规划求解该问题。设置3个商店*A*、*B*、*C*的编号分别为1～3。

这里总员工数为n=5，商店个数m=3，假设从商店3开始决策起。

设置二维动态规划数组为dp，其中dp[i][s]表示考虑商店i～商店m并分配s个人后的最优赢利。

另外设置二维数组pnum，其中pnum[i][s]表示求出dp[i][s]时对应商店i的分配人数。

从商店m→商店1反向求解

对应的状态转移方程如下：

dp[m+1][j]=0　　边界条件（类似终点的dp值为0）

dp[i][s]=max(v[i][j]+dp[i+1][s-j])　　pnum[i][s]=dp[i][s]取最大值的j（0≤j≤n）

显然，dp[1][n]就是最优赢利。

```
//问题表示
int m=3,n=5;                                        //商店数为m,总人数为n
int v[MAXM][MAXN]={{0,0,0,0,0,0},{0,3,7,9,12,13},
                   {0,5,10,11,11,11},{0,4,6,11,12,12}}; //不计v[0]行
//求解结果表示
int dp[MAXM][MAXN];
int pnum[MAXM][MAXN];
```

```
void Plan()                                    //求最优方案dp
{  int maxf,maxj;
   for (int j=0;j<=n;j++)                      //置边界条件
     dp[m+1][j]=0;
   for (int i=m;i>=1;i--)                      //i从商店3到1进行处理
   {  for (int s=1;s<=n;s++)                   //分配的总人数为s
      {  maxf=0;
         maxj=0;
         for (j=0;j<=s;j++)//找该商店最优情况maxf和分配人数maxj
         {  if ((v[i][j]+dp[i+1][s-j])>=maxf)
            {  maxf=v[i][j]+dp[i+1][s-j];
               maxj=j;
            }
         }
         dp[i][s]=maxf;
         pnum[i][s]=maxj;
      }
   }
}
```

员工数 / 商店	0人	1人	2人	3人	4人	5人
A	0	3	7	9	12	13
B	0	5	10	11	11	11
C	0	4	6	11	12	12

最优资源分配方案如下:
C商店分配3人
B商店分配2人
A商店分配0人
该分配方案的总赢利为21万元

补充：从商店1 → 商店m正向求解

dp[i][s]表示考虑商店1 ~ i并分配s个人后的最优赢利。

商店个数m，总员工数为n

dp[i][s]表示考虑商店1～i并分配s个人后的最优赢利。

对应的状态转移方程如下：

第0个阶段

dp[0][j]=0

dp[0][j]=0 边界条件（类似终点的dp值为0）

商店i选择j个人

dp[i][s]=max(v[i][j]+dp[i-1][s-j])

pnum[i][s]=dp[i][s]取最大值的j（1≤s≤n,0≤j≤s）

dp[i][s] = max(dp[i-1][s-j] + v[i][j]

pnum[

员工数 / 商店	0人	1人	2人	3人	4人	5人
A	0	3	7	9	12	13
B	0	5	10	11	11	11
C	0	4	6	11	12	12

最优资源分配方案如下:
C商店分配3人
B商店分配2人
A商店分配0人
该分配方案的总赢利为21万元